

Cross-Cloud Console Parity Audit

Date: 2026-05-28 **Authors:** Audit conducted post-Phase E (Azure roadmap A–E complete) **Scope:** Compare AWS, GCP, and Azure console surfaces in the Vyomi simulator across architecture, create flows, detail blades, rich-config sub-blades, and ancillary UX (action toolbars / activity / metrics / tags / notifications / space gate).

1. Executive Summary

Dimension	AWS	GCP	Azure	Leader
Services exposed	9	9	11	Azure
Standalone HTML console	No	No	Yes	Azure
Multi-step Create wizards	1/9	1/9	11/11	Azure
Sub-nav detail blade	9/9	7/9	11/11	Azure / AWS tied
Sub-blade items (total)	~50	~30	165	Azure
Rich-config sub-blades (real PUT/PATCH)	~18	~6	~37	Azure
Drill-down navigation w/ breadcrumbs	partial	partial	yes (3-level)	Azure
Lifecycle action toolbar	EC2, RDS	Compute, SQL	VM	tied
Per-resource activity log	partial (events)	partial (ops)	full (resource_id-tagged)	Azure
Metrics sparklines	placeholder	partial	yes (seeded)	Azure
Tags editor (uniform)	7/9	partial	11/11	Azure
Notifications drawer	No	No	Yes	Azure
Space-context gate	implicit (legacy AWS space)	implicit	explicit (requireAzureSpace)	Azure
Multi-step Provider→Space→Console flow	yes	yes	yes (Phase D fix)	tied

Headline. Azure console is **substantially ahead** in every dimension after Phases A–E. AWS is a close second on detail-blade depth and per-service customization (its sub-tabs are quite rich) but lacks the consistent wizard layer and the rich-config sub-blade pattern. GCP is a clear third — most surfaces are flat grids with simple dialogs; drill-down exists only for a few services.

2. Architecture Comparison

Aspect	AWS	GCP	Azure
Surface	React SPA inside <code>static/index.html</code> , view-based router	React SPA inside <code>static/index.html</code> , view-based router	Standalone <code>static/azure-console.html</code> reachable at <code>/console/azure</code>
Entry point	Providers → Spaces → Connect → <code>view='dashboard'</code>	Providers → Spaces → Connect → <code>view='gcp-console'</code>	Providers → Spaces → Connect → <code>window.location='/console/azure'</code> (gated by <code>requireAzureSpace()</code>)
Component model	Per-service React component (<code>EC2View</code> , <code>RDSView</code> , etc.) inline in <code>index.html</code>	Per-service React component (<code>GCPComputeView</code> , <code>GCPCloudSQLView</code> , etc.) inline in <code>index.html</code>	Single SPA with <code>SUB_RENDERERS</code> dispatch dict keyed by sub-blade type
Schema source-of-truth	Inline component code	Inline component code	<code>core/azure_wizards.py</code> + <code>core/azure_subblades.py</code> + <code>providers/azure_services.py::RESOURCE_CATALOG</code> — declarative, separable
Browser navigation	History via React state	History via React state	Standalone — full page; gate redirects on bad state
Tech stack	React 18 + Babel inline + manual <code>classNames</code>	Same	Vanilla JS + <code>el()</code> factory + CSS classes
Line count	~13 KLOC of mixed JSX inline	Same file as AWS, ~6 KLOC GCP-specific	3095 lines (HTML+CSS+JS) standalone

Implication. AWS/GCP per-service components were written each in isolation and grew organically — there's no shared "schema" that says "every service has a wizard with these tab types and these field types"; each service component reimplements its own form, modal, and table. Azure's declarative schema layer (`WIZARDS` + `SUB_BLADES` dicts) is the architectural unlock that made consistent portal-fidelity coverage tractable.

3. Service Catalog Coverage

AWS — 9 services

- Compute: **EC2**
- Storage: **S3** (buckets + objects)
- Security: **IAM**
- Databases: **RDS**, **DynamoDB**
- Messaging: **SQS**
- Integration: **API Gateway**
- Serverless: **Lambda**
- Networking: **VPC**

GCP — 9 services

- Compute: **Compute Engine**
- Storage: **Cloud Storage**
- Databases: **Cloud SQL**, **Firestore**
- Messaging: **Pub/Sub**
- Serverless: **Cloud Functions**
- Integration: **API Gateway**
- Networking: **VPC**
- Security: **IAM**

Azure — 11 services

- Compute: **Virtual Machines**
- Storage: **Storage accounts**
- Databases: **SQL servers**, **Cosmos DB**
- Messaging: **Service Bus**
- Serverless: **Function apps**
- Integration: **API Management**, **Event Grid topics** ← new in Phase E
- Networking: **Virtual networks**
- Security: **Entra ID / RBAC**, **Key vaults** ← new in Phase D

Notable gaps in AWS/GCP catalog vs Azure pattern:

- AWS missing: secrets manager (KMS / Secrets Manager), EventBridge (= Azure Event Grid), Step Functions
- GCP missing: KMS, Eventarc (= Azure Event Grid), Workflows

4. Create Flow Parity

Pattern	AWS	GCP	Azure
Multi-step wizard with tabs (Basics → ... → Tags → Review+create)	0/9	1/9 (Compute Engine has multi-section but no Review tab)	11/11
Single-form modal	8/9	8/9	0 (legacy <code>openFlatCreate</code> exists as fallback but never triggered — every service has a wizard)
Inline create panel	1/9 (IAM)	0	0
Conditional fields (<code>ifEquals</code>)	partial (RDS Easy vs Advanced mode)	no	yes (declarative — used 16 times across wizards)
Per-field regex validation	partial (some HTML5 patterns)	no	yes (every required field has <code>validate.regex</code>)
Per-tab error chips on stepper	n/a	n/a	yes
Auto Review+create tab	n/a	n/a	yes (auto-generated from schema)

Pattern	AWS	GCP	Azure
vmSize/machineType picker from catalog	yes (<code>api_ec2_amis</code> and instance types)	yes (machineTypes list)	yes (shared <code>/api/instances/catalog</code> → 74 SKUs across providers)
tagsEditor field type	partial (tag rows in some forms)	no (CSV labels)	yes (consistent <code>tagsEditor</code> field type)
subnetEditor field type	partial (VPC inline subnet list)	no	yes (<code>subnetEditor</code> field type in VNet wizard)

Azure wizard count: 52 tabs / 146 fields across 11 services (Phase B + E additions).

Asymmetry example. Creating a VM:

- AWS EC2: one modal with all fields stacked — name, AML, size, key pair, security group, subnet, storage, tags. ~12 fields.
- GCP Compute Engine: multi-section modal with collapsible Machine Configuration / Boot Disk / Networking / Security/Metadata / Launch Summary. ~18 fields. No validation chips, no Review.
- Azure VM: 8-tab wizard (Basics / Disks / Networking / Management / Monitoring / Advanced / Tags / Review+create) with per-tab validation, conditional fields (SSH-key picker only shown when auth=SSH), and a Review tab that lists every chosen value with Edit links back to the relevant tab. 33 fields.

5. Detail Blade Parity

Pattern	AWS	GCP	Azure
Per-resource detail blade	9/9	7/9 (Storage/VPC are flat-table-only)	11/11
Top-tabs (Overview / Tags / JSON)	yes (e.g., EC2 has 8 sub-tabs)	yes (most have Overview / Networking / Security)	yes (overview/json/tags rendered as universal <code>SUB_RENDERERS</code>)
Left sub-nav with grouped sections	no (top tabs only)	no (top tabs only)	yes (Settings / Operations / Monitoring groups + search filter)
Drill-down with breadcrumbs	partial (S3 bucket→objects only)	partial (Firestore collections→docs, Pub/Sub topics→subs, API Gateway 3-level)	yes (Cosmos databases→containers; SB topics→subs→rules; KV secrets→versions — all use <code>drillCrumbs</code> helper)
Sub-blade items per service (avg)	~5–6 (tabs)	~3–4 (tabs)	~15
Sub-blade items total	~50	~30	165
Implemented renderers	~50 (every tab is bespoke)	~30	37 (universal + per-service)

Pattern	AWS	GCP	Azure
Stub placeholders for parity	n/a	n/a	~32 ("Coming soon" cards matching real portal nav)
Activity log tab per resource	partial (RDS events log, Lambda invocations)	partial (Cloud SQL operations)	yes (universal <code>activityLog</code> renderer filtered by <code>resource_id</code>)
Metrics tab per resource	partial (placeholder text)	partial (CPU/memory footprint on Compute only)	yes (universal <code>metrics</code> renderer — 4-6 deterministic seeded sparklines per service)
Properties dump tab	no	no	yes (universal <code>properties</code> renderer — recursive walk into flat kv-grid)
Locks tab	n/a (AWS concept doesn't exist)	n/a	yes (universal stub)
JSON view tab	partial	yes (most have JSON dump)	yes (universal <code>json</code> renderer)

Visual proof — Azure VM detail blade left rail:

```

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
— Settings —
Networking
Connect
Disks
Size
Identity
Properties
Locks
— Operations —
Auto-shutdown
Backup
Extensions + applications
Run command
— Monitoring —
Insights
Alerts
Metrics

```

Equivalent AWS EC2 detail blade tabs:

```

Details · Status checks · Monitoring · Tags · Networking · Security · Storage · Console

```

AWS has the *content* (status checks tab, console embed) but in a flat 8-tab strip; Azure has 19 sub-blade items organized into 3 sub-nav groups. AWS readers don't get the portal-faithful "Operations" / "Monitoring" mental grouping.

6. Rich-Config Sub-Blade Parity (Real PUT/PATCH-Backed Editors)

A "rich-config sub-blade" is a sub-blade UI that **writes config to the backend** (not just lists). Counted by service:

Service	AWS rich-config editors	GCP rich-config editors	Azure rich-config editors
Compute (VM)	3 (SG rules, terminal exec, runtime sandbox status)	3 (startup script, shielded VM options, runtime sandbox)	6 (Networking, Disks, Size resize, Connect, Auto-shutdown, Metrics)
Object Storage	2 (bucket versioning, bucket notifications)	1 (bucket lifecycle / labels patch)	4 (Containers list, Access keys, Encryption, Lifecycle rules editor w/ tier transitions)
Relational DB	4 (RDS modify, snapshots, parameter group, subnet group)	2 (Cloud SQL connectivity, security flags)	6 (Databases drill, Firewalls editor , Connection strings, Pricing tier picker applying per-DB PATCH, Properties, Locks)
Messaging — queue	4 (SQS attributes, DLQ, send/receive, purge)	n/a	4 (Queues list, Shared access, Networking, Properties)
Messaging — topic	n/a	2 (Pub/Sub topic+sub updates, publish/pull)	6 (Topics→Subs→Rules drill-down with SqlFilter/CorrelationFilter , Shared access, Networking)
Document DB	2 (DynamoDB items, query/scan)	1 (Firestore doc edit)	5 (Cosmos databases→containers drill-down with partition key path , Replicate globally, Backup, Throughput RU/s, Properties)
Serverless	4 (Lambda code editor, configuration update, invoke test, permissions)	2 (Cloud Functions test invoke, env vars)	6 (Configuration, Authentication, Identity, Networking, Deployment slots w/ PUT, Deployment Center w/ 4 sources)
API Gateway	4 (Resources tree, Methods, Integrations, Stages)	2 (Configs, Gateways)	6 (APIs/Products/Subscriptions lists, Policies XML editor w/ 5 templates, Properties)
Network	5 (Subnets, SGs, Route tables, IGWs, SG rules editor)	1 (Firewall rules editor)	6 (Address space, Subnets, Connected devices, Peerings w/ remote VNet picker + real PUT , Properties)
Identity / RBAC	4 (IAM users, groups, roles, policy doc editor)	2 (Service accounts, project policy bindings)	1 (RBAC — minimal; portal flow is its own wizard)
Secrets / KMS	n/a (not exposed)	n/a (not exposed)	5 (Secrets→versions+rotation drill-down , Keys list, Certificates, Access policies w/ permission matrix editor , Networking)
Eventing	n/a (EventBridge not exposed)	n/a (Eventarc not exposed)	3 (Event subscriptions w/ 6 destination types + filter editor , Access keys, Networking)
TOTAL	~32 editors	~16 editors	~58 editors (37 implemented + 21 stubs for parity)

Azure has roughly 2x AWS's coverage and 4x GCP's at the rich-config layer. The gap is widest in:

- **Eventing** — AWS EventBridge + GCP Eventarc both have backend support in the simulator but no console at all
- **Secrets management** — neither AWS Secrets Manager nor GCP Secret Manager has a console
- **Drill-down editors** — only Azure has consistent multi-level drill (databases→containers, topics→subs→rules, secrets→versions)

7. Lifecycle Action Toolbar Parity

The "start/stop/restart" toolbar at the top of a VM/DB detail blade:

Action	AWS	GCP	Azure
VM Start	yes (EC2)	yes (Compute)	yes (VM)
VM Stop / Power off	yes	yes	yes
VM Restart / Reboot	yes	yes	yes
VM Deallocate (release host resources, keep disk)	no (terminate is destroy)	partial (stop = deallocated)	yes (separate from Power off)
Connect button → modal with SSH/exec commands	yes (terminal embed)	yes (terminal embed)	yes (modal with copy-able commands)
DB Start/Stop	yes (RDS)	yes (Cloud SQL)	n/a (SQL Server doesn't have stop in this catalog)
State-based disable	yes	yes	yes
Bulk action on grid (multi-select)	yes (EC2, RDS)	no	no (Azure portal doesn't either)

Verdict: full parity on lifecycle actions — Azure copied AWS's pattern when this was added in Phase A. AWS's multi-select bulk action is the one place AWS wins; not in portal so Azure doesn't replicate.

8. Ancillary UX Parity

Feature	AWS	GCP	Azure
Per-resource Activity log tab	partial (RDS event log, Lambda invocations)	partial (Cloud SQL operations table)	yes — universal, filtered from <code>/api/cloudsim/events</code> by <code>resource_id</code>
Per-resource Metrics tab	placeholder	partial (Compute CPU/memory only)	yes — 4-6 deterministic seeded sparklines per service type
Tags editor (KV grid, +Add)	yes for EC2/RDS/DynamoDB/SQS; missing for IAM/S3/Lambda/API GW/PC	partial — labels exist as CSV in create forms, no dedicated editor	yes — universal <code>tagsEditor</code> field type + Tags sub-blade
Top-bar notifications bell + drawer	no	no	yes — <code>NOTIFS</code> ring (50 entries), bell w/ red dot, slide-in drawer

Feature	AWS	GCP	Azure
Grid filter (free-text)	yes (most service grids)	partial	yes (universal <code>renderGrid</code> filter)
Sortable columns	partial	partial	yes (click-to-sort with arrow chip)
Resource breadcrumbs	partial (S3 bucket > object)	partial (Firestore, API GW)	yes (universal <code>drillCrumbs</code> helper used by 3 Azure drill-downs)
Resource detail "Essentials" card (resource group, location, sub id, status, SKU)	partial (per service)	partial	yes — universal at top of every <code>overview</code> renderer
Properties view (recursive flat kv dump of every property)	no	no	yes — universal <code>properties</code> renderer
JSON view (raw record)	partial	partial	yes — universal <code>json</code> renderer
Search across resources (top bar)	yes ("global search" input)	yes	placeholder only — no live search
Cloud Shell button	yes (opens terminal in EC2/Compute)	yes	placeholder (top-bar icon, no impl)

9. Space-Context Gate

Surface	Gate behavior
AWS dashboard	Reachable only via SPA state machine — <code>useState('providers')</code> boots to providers view. The legacy <code>space-legacy</code> is always <code>provider=aws</code> , so AWS works without ever calling <code>/api/spaces/.../switch</code>
GCP console	Same — SPA-state-only gating; relies on whichever space the SPA thinks is active via <code>localStorage</code>
Azure console	Standalone HTML → needs explicit gate. Phase D added <code>requireAzureSpace()</code> that fetches <code>/api/spaces/active</code> , redirects to <code>/ui</code> with explainer if no active space OR active space is not <code>provider=azure</code> . Phase D also added the Provider→Space→Console flow to the SPA: clicking Connect on an Azure space card now POSTs <code>/api/spaces</code> + <code>/api/spaces/{id}/switch</code> before navigating to <code>/console/azure</code>

Why Azure needed the explicit gate. The state proxy (`_SpaceScopedDictProxy`) rejects writes when `space.provider !== REQUEST_PROVIDER` — silently routes to a per-request scratch dict that never persists. AWS/GCP can ride on the always-AWS-default `space-legacy`; Azure has no such default, so direct navigation to `/console/azure` without a switched space leads to a broken (writes-vanish) experience. The gate prevents that.

Cross-cloud gap. GCP can be reached the same way today — if a user (or test) sets `cloudlearn.active.space.provider=gcp` in `localStorage` and the backend is still on AWS-legacy, GCP writes will

exhibit the same scratch-dict trap. The gap was just never noticed because the SPA's `view='gcp-console'` is always entered via Connect (which sets localStorage). A direct curl to `/api/.../<gcp-resource>` would have the issue.

10. Per-Service Deep-Dive Parity

Compute (VM)

Capability	AWS EC2	GCP Compute	Azure VM
Create wizard	flat modal	5-section modal	8-tab wizard w/ validation
Size picker from catalog	yes	yes	yes
Action toolbar	Start/Stop/Reboot/Terminate	Start/Stop/Reset/Delete	Start/Stop/Restart/Deallocate/Connect
Connect modal	inline terminal	inline terminal	modal w/ SSH/exec cmds + copy
Networking sub-blade	Networking tab	Networking tab	Networking sub-blade
Disks sub-blade	Storage tab	implied	Disks sub-blade (OS + data)
Size sub-blade (resize)	no	partial (machine type change)	yes (real PATCH on hardwareProfile.vmSize)
Auto-shutdown	no	no	yes (config view + form)
Boot diagnostics	partial	partial	yes
Host-clamp warning when oversized	n/a	n/a	yes (Phase A heritage)
Real LXD/multipass backing	yes (Phase A)	yes (Phase A)	yes (Phase A)

Object Storage

Capability	AWS S3	GCP Cloud Storage	Azure Storage
Bucket detail blade	none (drills to objects)	minimal	full 5-group sub-nav
Versioning editor	yes	yes (PATCH)	yes (in Lifecycle / Data protection sub-blade)
Lifecycle rules editor	no	yes (basic, in patch_bucket)	yes — full rules-with-transitions editor (Phase D)
Notifications (event grid)	yes (event bridge / SNS / SQS / Lambda triggers)	no	partial (Event Grid is a separate service)
CORS / access keys	partial	partial	yes (Access keys sub-blade)

Capability	AWS S3	GCP Cloud Storage	Azure Storage
Encryption (MMK/CMK)	n/a	partial	yes (Encryption sub-blade)

Relational Database

Capability	AWS RDS	GCP Cloud SQL	Azure SQL
Multi-engine selector	yes (PG/MySQL/MariaDB)	yes (PG/MySQL/SQL Server)	implicit (Azure SQL only — Cosmos covers NoSQL)
Easy/Advanced create	yes	no	n/a (single wizard w/ optional fields)
Snapshots	yes (sub-tab)	yes (Backups)	partial (stub)
Parameter groups	yes (sub-grid)	partial (flags)	partial (stub)
Firewall rules	partial (in security groups)	partial (authorized networks)	yes — firewalls sub-blade with <code>listChildResources</code>
Pricing tier picker	n/a	n/a	yes (Phase D) — DTU+vCore matrix, applies per-DB via PATCH
Connection strings	partial	yes (rendered in detail)	yes — ADO.NET/JDBC/ODBC/PHP forms

Document Database

Capability	AWS DynamoDB	GCP Firestore	Azure Cosmos DB
Item/document editor	yes (JSON inline)	yes (field editor)	implied via Data Explorer drill
Query builder	yes	yes (runQuery)	not yet (no inline query)
Partition key declaration	yes (at create)	implicit	yes — drill-down to container shows + editable PK paths (Phase E)
Throughput / autoscale	yes (PAY_PER_REQUEST / PROVISIONED toggle)	n/a	yes — manual/autoscale toggle w/ RU/s slider (Phase D)
Global distribution	partial	n/a	yes — Replicate globally sub-blade
Backup policy	partial	partial	yes — Backup & Restore sub-blade

Serverless

Capability	AWS Lambda	GCP Cloud Functions	Azure Function App
Code editor	yes (inline textarea)	yes (inline)	not yet inline (uses deploy flow)

Capability	AWS Lambda	GCP Cloud Functions	Azure Function App
Test/Invoke	yes (with payload + types)	yes (call)	not yet inline
Environment variables	yes	yes	yes (Configuration sub-blade)
Resource-based permissions	yes (statement editor)	n/a	partial (Authentication sub-blade)
Versions	yes (list + publish)	yes (versions sub-grid)	partial (Slots covers this)
Deployment slots	n/a (Lambda has aliases instead)	n/a	yes — list + Add slot w/ clone-from-prod (Phase D)
Deployment center w/ multi-source	n/a	n/a	yes — Zip / GitHub Actions / Local Git / Container (Phase E)
Logs / invocations	partial	yes (invocation history)	partial (stub)

Messaging — queue

Capability	AWS SQS	(GCP n/a)	Azure SB queues
FIFO/Standard toggle	yes	n/a	partial (catalog has Basic/Std/Premium)
Send message form	yes	n/a	partial (uses SDK paths)
Receive message form	yes (w/ visibility timeout)	n/a	partial
DLQ config	yes	n/a	yes (in subscription form)
Purge	yes	n/a	partial

Messaging — topic

Capability	(AWS SNS n/a)	GCP Pub/Sub	Azure Service Bus topics
Topic list	n/a	yes	yes
Subscriptions per topic	n/a	yes (sub-grid)	yes — drill-down (Phase E)
Filter rules / message routing	n/a	basic (filter expr on subscription)	yes — drill 3-level: topic → sub → rule w/ SqlFilter or CorrelationFilter (Phase E)
Publish/pull test UI	n/a	yes	partial

Integration / API Gateway

Capability	AWS API Gateway	GCP API Gateway	Azure APIM
API/Resource/Method tree	yes (3-level)	yes (3-level APIs→Configs→Gateways)	partial (3 list sub-blades)
Method integrations (MOCK/HTTP/AWS_PROXY)	yes	n/a (OpenAPI spec only)	partial
Stages	yes	yes (Gateways)	partial
Policy XML editor	n/a (uses Gateway responses)	n/a	yes — 5 templates: empty/rate-limit/cors/jwt-validate/set-backend (Phase E)
Subscriptions / Products / Developers	n/a	n/a	yes — APIM-specific concept

Network / VPC

Capability	AWS VPC	GCP VPC	Azure VNet
VPC/VNet list	yes	yes	yes
Subnets editor	yes (inline form)	yes	yes (subnetEditor field in wizard)
Security groups / firewall rules	yes (with rules editor)	yes (firewall rules editor)	partial (NSG is a separate not-yet-exposed service)
Route tables / routes	yes (RT + add route + associate subnet)	n/a (BGP)	partial (stub — not portal-faithful)
Internet gateway	yes	n/a (cloud default)	n/a (not in catalog)
Peerings editor w/ real PUT	partial (VPC peering exists in catalog)	n/a	yes (Phase D) — Add peering modal with remote VNet picker + 4 flags
Address space view	partial	partial	yes
Connected devices	partial	partial	partial (stub)

Identity / RBAC

Capability	AWS IAM	GCP IAM	Azure RBAC + Entra
Users/Groups/Roles	yes (inline create)	partial (service accounts only)	partial (role assignments)
Policy document editor	yes (JSON)	partial	n/a (uses scope-based)
Policy attachment matrix	yes (target type selector)	yes (policy bindings)	yes (Access control sub-blade lists scoped assignments)

Capability	AWS IAM	GCP IAM	Azure RBAC + Entra
Identity providers (SAML/OIDC)	yes	partial	n/a
Password policy editor	yes	n/a	n/a
Service principal management	yes	yes (svc accts)	partial

Secrets / Eventing (Azure-only territory)

Capability	(AWS Secrets Manager: n/a in console)	(GCP Secret Manager: n/a in console)	Azure Key Vault + Event Grid
Secrets list + versioning	n/a	n/a	yes — drill-down to versions + rotation policy (Phase E)
Access policies (per-principal perms)	n/a	n/a	yes — Add policy modal w/ 15+8+17 perm checkboxes (Phase D)
Event topics + subscriptions	n/a	n/a	yes — Event Grid as 11th service (Phase E)
Multi-destination subscriptions (WebHook / Queue / Function / EventHub)	n/a	n/a	yes (Phase E)

11. What Brings AWS to Azure Parity

Ranked by user-visible impact / effort ratio:

1. 🔥 **Multi-step Create wizards for all 9 services** — biggest UX gap. Build an AWS equivalent of `core/azure_wizards.py` (schema-only module) + a renderer in `static/index.html` that consumes it. Estimated 2-3 weeks for parity across EC2, RDS, S3, DynamoDB, SQS, Lambda, API Gateway, VPC, IAM.
2. 🔥 **Sub-nav detail blade with grouped sections** — currently AWS uses flat top-tabs (8 max). Add a left-rail + content-pane layout per service with Settings / Operations / Monitoring groupings. Reuses Azure's `SUB_RENDERERS` pattern.
3. **Standalone /console/aws HTML** — frees the AWS surface from sharing state with the SPA shell, matches the dedicated-console mental model the real AWS console has.
4. **Per-resource activity log tab** — backend already records events via `_record_usage` ; just need a universal renderer.
5. **Tags editor universalization** — every service that lacks it (IAM, S3, Lambda, API GW, VPC) gets a `tagsEditor`.
6. **Notifications drawer** — 50-entry ring + bell + red dot, mirrors Azure pattern.
7. **EventBridge service** — adds long-needed eventing surface.
8. **Secrets Manager service** — adds long-needed secrets surface.
9. **KMS service** — adds key management surface.

12. What Brings GCP to Azure Parity

Ranked by user-visible impact / effort ratio:

1. 🔥 **Detail blade for Cloud Storage and VPC** — currently these are flat tables with no per-resource detail. Add a `GCPStorageBucketView` and `GCPVPCNetworkView` component each.
2. 🔥 **Multi-step Create wizards for all 9 services** — only Compute Engine has even a multi-section modal today, and that one lacks a Review tab. Schema-driven wizard module same as Azure's pattern.
3. 🔥 **Sub-nav detail blade with grouped sections** — same as AWS.
4. **Per-service rich-config editors** — GCP has only 6 today vs Azure's 37. Specific high-value adds: Cloud SQL flags editor, Pub/Sub subscription filter editor, Cloud Storage lifecycle, Cloud Functions env vars + trigger editor.
5. **Drill-down for Cloud Storage** — bucket → objects → object detail.
6. **Per-resource activity log tab** — backend has operations logs for Cloud SQL and Compute; surface universally.
7. **Tags / labels editor** — replace CSV input fields with a real kv grid like Azure.
8. **Notifications drawer.**
9. **Secret Manager service** — backend exists but no console.
10. **Eventarc service** — backend exists but no console.

13. Cross-Cloud Pattern Library (Reusable from Azure → AWS/GCP)

These Azure-side patterns are provider-agnostic and could be lifted into AWS/GCP work:

Pattern	Azure location	Reuse approach
Wizard schema (tabs/sections/fields with validation + <code>ifEquals</code>)	<code>core/azure_wizards.py</code>	Create <code>core/aws_wizards.py</code> + <code>core/gcp_wizards.py</code> with same shape; SPA renderer can be one shared component if SPA → standalone-HTML refactor happens, or a React-port of <code>openWizard</code> for the current SPA
Sub-blade schema (grouped left-nav with type-dispatch renderer)	<code>core/azure_subblades.py</code>	Same pattern per provider; rename <code>SUB_RENDERERS</code> registry to share renderers across providers where logic is identical (Tags, JSON, Properties, IAM)
<code>requireAzureSpace()</code> gate	<code>static/azure-console.html:503</code>	Generalize to <code>requireProviderSpace(provider)</code> ; apply to standalone consoles per provider
<code>recordNotif + drawer</code>	<code>static/azure-console.html:275–305</code>	Move to a shared <code>static/console-common.js</code>
<code>drillCrumbs</code> helper	<code>static/azure-console.html</code>	Same — extract to shared
Metrics sparkline (seeded SVG)	<code>SUB_RENDERERS.metrics</code>	Same — seed by <code>series-label</code> + <code>resource-id</code> for deterministic shapes

Pattern	Azure location	Reuse approach
<code>ListChildResources</code> + empty state	<code>static/azure-console.html</code>	Same — generalize for AWS sub-resources (e.g., SG rules under a VPC)
VM action toolbar (state-based disable)	Phase A pattern	Already mirrored in AWS EC2 grid actions; Azure pattern adds Connect modal + Deallocate semantics
Per-resource activity log via <code>resource_id</code> -tagged events	Phase C pattern	Bump AWS/GCP <code>_record_usage</code> calls to include <code>resource_id</code> ; reuse Azure's <code>fetchActivity</code> helper
Catalog endpoint w/ wizard + subBlades	<code>providers/azure_services.py::catalog_for_console</code>	Mirror for AWS/GCP: single endpoint returning all metadata so the SPA can render any service generically

14. Conclusion

The Azure console after Phases A–E is **the de-facto reference implementation** for what a portal-faithful, schema-driven simulator console looks like. AWS has deep per-service customization but inconsistent UX patterns; GCP has the thinnest surface of the three (most surfaces are flat grids).

The fastest path to cross-cloud parity is to **lift Azure's schema-driven architecture across the board**:

1. Define `WIZARDS` and `SUB_BLADES` dicts per provider.
2. Make the catalog endpoint per-provider mirror Azure's (return wizard + subBlades alongside `createFields`).
3. Refactor the SPA's AWS/GCP rendering into a generic `openConsole(provider)` that takes the catalog payload and produces the portal-style two-pane layout — instead of 18+ bespoke React components.

That refactor is in the 4–6 week range for both AWS and GCP combined. It would also unlock future provider additions (Oracle Cloud, IBM Cloud, etc.) at a fraction of today's per-service effort.

See companion memory files: [\[\[azure-console-phase-a\]\]](#) through [\[\[azure-console-phase-e\]\]](#) for the Azure roadmap details, and [\[\[appliance-static-rebuild-loop\]\]](#) for the dev-loop reminder.